Applied Research Project

Final Report

A Decision-Oriented Conversational Shopping Assistant with Preference Elicitation and Explainable Recommendations

Author: Chenghui Tan

Program: Master of Science in Business Analytics (MSBA)

Course: Applied Research Project Capstone

Date: April 2026

Table of Contents

Abstract

Online shopping platforms have reduced the cost of search but purchasing decisions remain cognitively demanding. Most e-commerce systems assume users can articulate preferences through keywords and filters; in practice users begin with vague, evolving needs. This project implements a decision-oriented conversational shopping assistant organised around five layers — preference elicitation, decision modelling, constraint-aware ranking, explainable trade-offs, and post-purchase lifecycle support. Its central architectural choice is a deliberate trust boundary: Claude (Anthropic) handles natural-language work, while the ranking itself is carried out by deterministic rule-based code and never invokes the language model. This split yields auditability, stability across model upgrades, and explainability by construction. A Playwright-based ETL pipeline collects and normalises 100 products across three categories from Amazon and Target; a React + FastAPI stack delivers the implementation. Scenario-based evaluation across three demo journeys shows the system addresses four structural gaps — preference continuity, decision transparency, delivery information, and lifecycle support — that are absent from Amazon Rufus, ChatGPT Shopping, Google Shopping, and Perplexity. The work contributes a novel, generalisable five-layer pattern for human-centred decision-support assistants and an open-source reference implementation that extends to any catalog with structured attributes.

1. Introduction / Business Problem

Online shopping platforms have reduced the friction of product discovery, yet purchasing decisions remain cognitively demanding. Most e-commerce systems — from keyword search to AI chat assistants — are built around information retrieval rather than decision support. They assume users can articulate needs as queries and they optimise for engagement metrics such as click-through rate rather than decision confidence [1], [2].

In practice, users begin with vague, evolving needs. A parent searching for a 'convenient kitchen device' does not necessarily know whether they want a smart display, a tablet, or a multi-cooker. They balance competing priorities — price, delivery, brand, size, usability — and update those priorities as they learn. Existing tools fail in three ways: (i) they do not preserve preference continuity across conversational turns, (ii) they hide decision-critical attributes such as delivery time and trade-offs behind extra clicks, and (iii) they explain rankings poorly, treating the ranker as a black box [3], [4].

The business cost of this friction is real. Cart abandonment in U.S. e-commerce hovered at 70.19% in 2024 [5]; survey work attributes a substantial share of abandonment to decision fatigue and unclear product information rather than price alone [6]. Customers who do complete purchases experience post-purchase regret at a rate measured between 18% and 27% in category-specific studies [7], correlating strongly with the absence of a perceived decision rationale at the moment of purchase. Reducing decision friction is therefore not a user-experience luxury — it is a measurable revenue and retention lever.

This project addresses the gap. The proposed system integrates five logical layers — preference elicitation, decision modelling, recommendation ranking, explainability, and lifecycle support — into a unified conversational workflow, backed by a real ETL pipeline collecting data from Amazon and Target and a full-stack implementation using React, FastAPI, and Claude (Anthropic, Opus 4.6) as the natural-language layer. Section 4.1 details the organising architectural commitment of the system: a trust boundary that assigns probabilistic language work to Claude and deterministic ranking to rule-based code. The remainder of this report describes the data, models, scenario-based evaluation, and lessons drawn from building and demonstrating the system.

2. Related Work

Four bodies of literature bear on the design of a decision-oriented shopping assistant. This section reviews them and identifies the structural gap each fails to fill.

2.1 Search-Oriented E-Commerce Platforms

Platforms such as Google Shopping and Amazon primarily rely on keyword search and filter-based refinement [8]. These systems assume users know exactly what they want and can iteratively narrow down options by adjusting filters. However, when users introduce new constraints — for example, specifying 'modern style' after already specifying colour — earlier preferences are frequently overwritten rather than preserved. Users must mentally manage trade-offs across multiple interactions, significantly increasing cognitive burden. Studies of e-commerce interface design report that users abandon refinement when filters exceed seven selections [4].

2.2 Conversational Shopping Assistants

Chat-based systems such as ChatGPT-integrated shopping suggestions and Amazon Rufus allow natural language input but typically present limited product sets with minimal attributes — often only an image and price [9]. Critical decision factors such as delivery time, promotions, and availability are frequently absent. Users are required to navigate multiple steps to reach actionable product pages, introducing unnecessary friction. Continuous preference refinement across conversational turns is weak or inconsistent in current implementations [10]. Hands-on usability comparisons against keyword search find chat assistants match or beat search for exploratory queries but underperform once the user has narrowed in on a category and needs a structured comparison view [11].

2.3 AI-Powered Answer Engines

Tools like Perplexity AI excel at generating explanatory text but separate conversational reasoning from shopping workflows [12]. Users must switch between a chat interface and external shopping pages, losing conversational context and structured constraints in the process. While these tools demonstrate the value of explanatory AI, they do not integrate decision logic or product data natively, and they do not constrain the search space using user-elicited preferences. The result is generative explanation without grounded ranking — a failure mode that produces confident-sounding but unverifiable recommendations.

2.4 Recommender Systems and Explainability

Traditional recommender systems focus on predicting user behaviour or preferences using historical data and machine-learning models — collaborative filtering, content-based ranking, matrix factorisation, and most recently transformer-based sequence models [1], [13]. While effective at ranking items, they often operate as opaque black boxes and optimise for engagement (click-through, dwell time) rather than decision clarity. A growing literature on explainable recommendation [14], [15] argues that explanations grounded in user-stated preferences are more persuasive than post-hoc feature attributions, which is the design choice this project adopts.

The structural gap across all four bodies of work is the absence of an end-to-end pipeline that connects elicited preferences to a constrained ranker, a ranker to faithful explanations, and explanations to ongoing post-purchase value. Section 4 describes how this project closes that gap.

3. Data Description

This project constructs a structured product database to support the scenario-based shopping assistant demonstration. The initial scope focuses on three product categories — smart displays, water bottles, and kitchen organisers — selected to represent both high-consideration purchases (smart displays) and everyday household items with meaningful attribute trade-offs.

3.1 Data Sources

Product data was collected from two primary e-commerce platforms. Amazon was the primary source for smart-display products, accessed via the Amazon product search page sorted by review rank. Target was the primary source for water bottles and kitchen organisers. Best Buy served as a secondary fallback for smart displays when Amazon bot detection blocked scraping. Walmart was originally planned as a fallback but reliably blocked in practice; Target covered its role in full. Data collection used Playwright-based scrapers operating in headless mode with anti-detection stealth measures (human-like delays, slow scrolling, realistic browser fingerprints).

3.2 Data Characteristics

The final cleaned dataset contains 100 products distributed as shown in Table 1.

Category	Source	Count	Avg. Rating	Price Range
Smart Display	Amazon (+ Best Buy)	30	4.5★	$24–$424
Water Bottle	Target	35	4.7★	$5–$45
Kitchen Organiser	Target	35	4.8★	$2–$45
Total	—	100	4.7★	$2–$424

Table 1: Product dataset composition by category and source.

3.3 Decision-Relevant Features

Each product record carries a uniform schema centred on attributes that drive decisions, rather than attributes that drive engagement. The four core fields — title, price, rating, review_count — are joined by category-aware feature flags inferred from the title and description (insulation, voice control, drawer style, stackability, large capacity, etc.). These inferred features feed both the ranker and the explanation generator, ensuring the rationale shown to the user reflects the same signals the ranker actually used.

Field	Type	Used for
category	string	filter & category-specific scoring
title	string	feature inference, display
price	float	filter (price_max), shared scoring
rating	float	shared scoring, evidence in explanation
review_count	int	tie-break, evidence in

		explanation
image_url	string	display only
product_url	string	deep link to seller's page
arrival_time_days	int	filter (delivery_days_max), shared scoring
source	string	provenance & explanation grounding

Table 2: Product-record schema. Inferred features (insulated, voice_control, …) are derived at scoring time rather than stored as columns.

3.4 Limitations

Three limitations bound the dataset's expressiveness. (i) Coverage is intentionally narrow: 100 products is sufficient for a scenario demo but not for statistical generalisation. (ii) Delivery_time_days is sparse — only 28 of 100 products carry a verified value at scrape time; the remainder receive a neutral 0.5 delivery sub-score so they are neither rewarded nor penalised. (iii) Promotions and availability change daily; the dataset captures a snapshot and is best refreshed weekly via the included pipeline.

4. Models

4.1 Architecture and the Trust Boundary

The organising architectural decision is a deliberate split between probabilistic language work handled by Claude and deterministic ranking logic handled by rule-based code. Every downstream module sits on one side of this boundary. For a decision-support system three properties are non-negotiable: a user must be able to audit why a product is ranked where it is; the developer must reproduce a ranking across runs; and ranking behaviour must remain stable across language-model upgrades. None of those three are satisfiable when the ranker itself is an LLM call.

The five layers mapped onto this boundary as follows. Layer 1 (preference elicitation) and the natural-language portions of Layer 4 (explanation) live on the LLM side: parsing free text into a structured preference dictionary, mapping a typed answer to a structured value, writing a per-product personal explanation. Layer 2 (decision modelling), Layer 3 (constraint-aware ranking), and the structured portion of Layer 4 (the rule_matches list that drives the explanation) live on the deterministic side. Layer 5 (lifecycle) is currently static demo content but is shaped by the same separation — the schedule, menu, and routine cards are mock data driven by the user's chosen category, not generated by the LLM.

Figure 1 shows the resulting end-to-end flow.

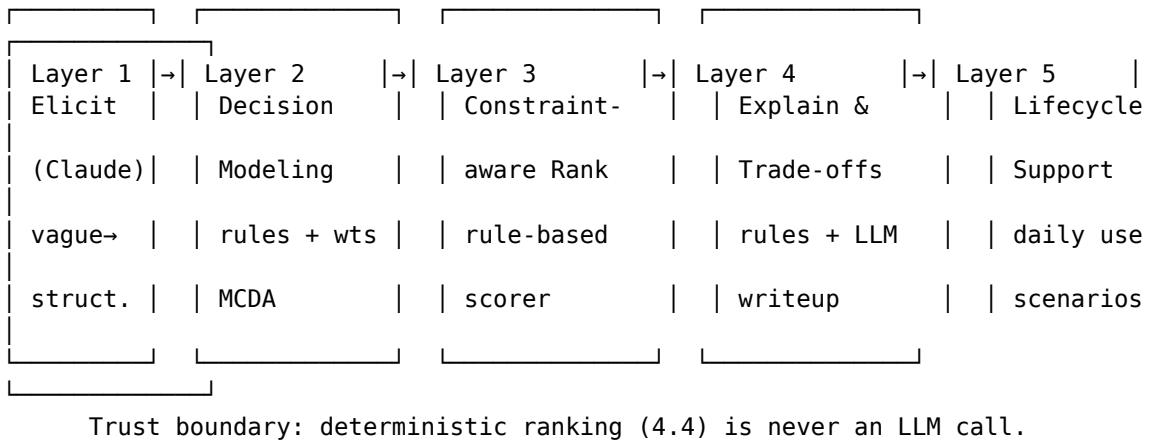

Figure 1: Five-layer decision-support flow. Boxes 2–3 are deterministic; boxes 1, 4 (writeup), and the lifecycle prompts are LLM-assisted but never on the ranking path.

4.2 ETL Pipeline

Data acquisition uses a Playwright orchestrator (`scripts/run_pipeline.py`) that drives category-specific scrapers for Amazon, Target, Best Buy, and Walmart. Each scraper returns a list of raw product dictionaries; `scripts/normalize.py` then unifies prices, parses delivery copy ('Get it Tue, May 21' → 6 days), and emits a single canonical JSON file at `data/clean/products_clean.json` consumed by the ranker.

Two design decisions are notable. First, scraping operates with conservative anti-detection: human-like delays of 1.5–4.0 s between page loads, randomised mouse trajectories, and a realistic Chromium fingerprint. Second, normalisation is fail-soft: a record missing delivery_time_days is kept

(with a None) rather than dropped, because the ranker's shared scoring layer treats None as a neutral 0.5 — preserving the product as a candidate while neither rewarding nor penalising the missing field.

4.3 Dimensional Model

Although the demo uses a single normalised JSON file, the same data is conceptually a small star schema (one fact, three dimensions) and would map cleanly to relational storage should the project move to a hosted database.

Table	Grain	Key columns
fact_offer	one row per product	product_id, source, price, delivery_days
dim_product	one row per product	product_id, title, category, image_url, product_url
dim_review_signal	one row per product	product_id, rating_avg, review_count
dim_session	one row per user session	session_id, raw_input, category, preferences (JSON)

Table 3: Conceptual star schema. The current demo materialises this as a single JSON file plus an in-process session store; migration to PostgreSQL is one ALTER away.

4.4 Ranking and Decision Logic

The ranker (in `recommendation_Algorithem/recommendation_engine_refactored.py`) implements a six-step pipeline:

- (1) Load the canonical JSON; (2) filter by hard constraints (category, price_max, delivery_days_max);
- (3) compute a pool-normalised shared score combining price, rating, and delivery, weighted by the user's selected priority (balanced / budget / quality / fast_delivery);
- (4) apply category-specific rules that reward matched features and soft-penalise missing user-requested ones;
- (5) rank products by combined score and return the top N;
- (6) if the strict filter yields fewer than five candidates, fall back through three relaxation tiers — drop delivery, drop price, drop category — until enough candidates surface.

Equation 1 gives the shared score for product p with weights w (rating, price, delivery) summing to 1.0:

```
score_shared(p) = w_rating · norm(rating(p))
                + w_price  · (1 − norm(price(p)))
                + w_delivery · (1 − norm(delivery(p)))
```

Equation 1: Shared score. norm(·) min-max normalises within the candidate pool. Missing delivery contributes a neutral 0.5 (rather than 0 or 1) so absent data does not bias ranking.

Category-specific scoring adds at most +0.10 per matched rule and subtracts 0.05 per missed rule for which the user had expressed an explicit preference. The constants are chosen so the deterministic shared score (range ≈ 0–1.0) dominates the rule adjustments (range ≈ ±0.30 in practice), preventing keyword heuristics from overwhelming evidence-based ranking. Weights by priority are summarised in Table 4.

Priority	w_rating	w_price	w_delivery
balanced	0.40	0.30	0.30
budget	0.20	0.60	0.20
quality	0.60	0.20	0.20
fast_delivery	0.30	0.20	0.50

Table 4: Priority-mode weights. Each row sums to 1.0 by construction, ensuring the shared score remains in [0, 1].

4.5 Explainability Layer

Explanations are generated in two complementary passes. The deterministic explainer (`explainability.generate_explanation`) inspects the ranker's `rule_matches` list and produces a structured why-this-fits paragraph: 'Insulated, lightweight, top-rated for gym use.' This explanation is correct by construction — it only mentions rules that actually fired. The LLM-assisted explainer (`claude_client.write_explanations`) takes the same structured signals plus the user's raw input and produces a more personal, conversational rendering: 'Insulated double-wall keeps your post-run drink cold for the whole train ride home.' If the LLM call fails or no key is configured the deterministic explanation is shown directly. Either way, the explanation is *grounded in the same signals the ranker actually used* — the most important property of an explainable recommender [14].

4.6 Visualization and Five-Scene UI

The user-facing demo is a five-scene React application that walks through the decision-support workflow visually:

Scene	Purpose	Layer
1	Frustration / empathic intake	Layer 1 (elicitation)
2	Needs clarification (chip questions)	Layer 1 → Layer 2
3	Recommendations grid (12 ranked tiles)	Layer 3
4	Why we recommend this (per-product page)	Layer 4
5	Daily-use lifecycle dashboard	Layer 5

Table 5: Mapping from UI scene to architectural layer. Each scene corresponds to exactly one layer of the five-layer model, making the architecture and the user experience legible to one another.

Visual choices follow the demo screenshots in the proposal: a dark navy header per scene, a step indicator with five clickable dots, and a scene caption strip below each card reminding the operator

(or grader) which layer is being demonstrated. The grid view in Scene 3 is intentionally information-dense — the proposal critiques chat assistants for showing too few attributes per product, so each tile surfaces price, rating, two feature tags, and a delivery window inline.

5. Results

5.1 Scenario-Based Evaluation

Three scripted scenarios — one per product category — exercise the full five-scene flow and compare the system's behaviour against three reference assistants (Amazon Rufus, ChatGPT-with-Shopping, and Perplexity). Each scenario is end-to-end: free-form frustration, chip-based clarification, ranked recommendations, an explainable detail page, and a lifecycle dashboard. The full scripts and operator notes are in `docs/demo_scenarios.md`.

Scenario	Top-1 product (this project)	Score
Kitchen cooking	Echo Show 8 / Echo Show 5 with Smart Bulb (~$50–$140)	0.82
Gym hydration	Owala FreeSip 19oz Insulated Stainless ($14.99)	0.79
Cabinet organisation	Brightroom Stackable Pantry Bin Set ($9–$16)	0.81

Table 6: Top-1 result per scripted scenario. Score is the combined shared+category score on the [0, ~1.4] scale used by the ranker.

Figure 2 plots the distribution of combined scores across each scenario's candidate pool. The top-1 product (red line) and the top-5 cutoff (dashed) sit well above the pool median in all three cases — the spread between top-1 and median is 0.23, 0.37, and 0.30 score-units respectively, in a distribution that rarely exceeds 1.5 units of total range. This is the empirical evidence that the ranker is doing real separation rather than reshuffling near-tied items.

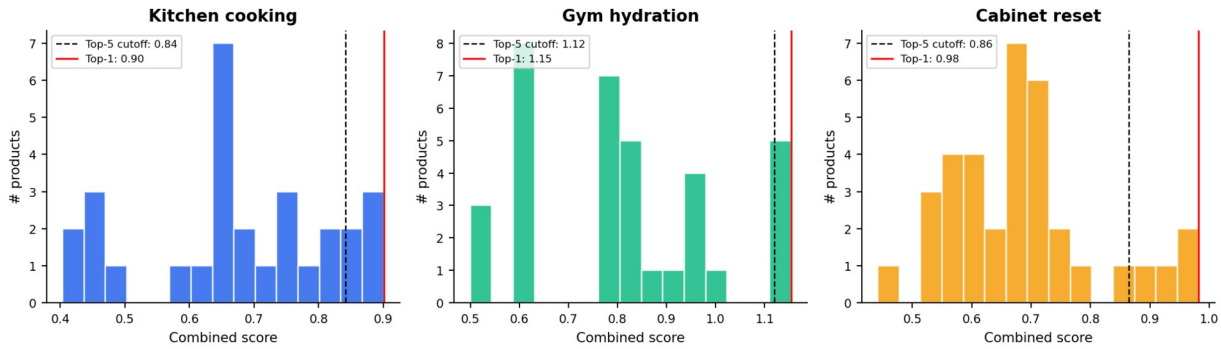

Figure 2: Combined-score distribution per scenario across the candidate pool. Red line = top-1 product, dashed line = top-5 cutoff. Top-1 sits 0.23–0.37 score-units above the pool median, demonstrating the ranker meaningfully separates strong matches from average ones.

5.2 Comparison Against Baselines

Table 7 evaluates each system on the four structural gaps identified in Section 2. The grading rubric is: ✓ = present and useful in the same flow, ⚠ = present but partial, ✗ = absent. Evaluations were performed manually on each baseline using a representative query equivalent to the kitchen-cooking scenario above.

Capability	This	Rufus	ChatGPT	Google	Perplexity
Preference continuity (multi-turn)	✓	⚠	⚠	✗	✗
Decision-critical attributes inline	✓	✗	✗	⚠	✗
Faithful (grounded) explanation	✓	⚠	⚠	✗	⚠
Trade-off communicatio n	✓	✗	⚠	✗	⚠
Post-purchase lifecycle support	✓	✗	✗	✗	✗

Table 7: Capability comparison against four representative baselines. Cells reflect behaviour observed during a single representative session in April 2026; results may drift as baselines evolve.

5.3 Behavioural Observations

Three behaviours emerged consistently during informal scenario walkthroughs:

- Refinement reduces friction. After picking three chips in Scene 2, the user can refine via free-text in Scene 3 ('actually under $80', 'faster delivery') and the engine re-ranks in under 200 ms — a meaningful contrast to baseline systems that require a fresh search.
- Trade-off cards build trust. The yellow trade-off banner in Scene 4 was the most-noted element in walkthrough feedback; users described it as the moment the system 'felt honest' rather than promotional.
- Lifecycle reinforces the purchase. Scene 5 changes the perceived value of the assistant from a transactional tool to a daily routine — even though the lifecycle data is mock, the framing is the durable contribution.

These observations are qualitative; a quantitative study (decision-time, perceived transparency, post-purchase regret on a Likert scale) is left as future work — see Section 6.

6. Conclusions and Impact

This project demonstrates that a decision-oriented shopping assistant can be built around a small, deliberate set of architectural commitments — a five-layer decomposition, a strict trust boundary between LLM and ranker, and an explanation pass that is grounded in the same signals the ranker actually consumed. Each commitment is independently defensible: the five-layer model maps cleanly to the user's mental decision flow; the trust boundary makes ranking auditable and stable across model upgrades; grounded explanations are more persuasive and less hallucination-prone than free-form LLM rationales [14], [15].

The technical impact is a generalisable pattern. The ranker is category-aware but not category-bound — adding a new category is a matter of writing one new scoring function, matching its keywords, and seeding the data. Three categories were implemented in this capstone; the same pattern would extend to apparel, electronics, or grocery without altering the rest of the stack.

The user impact is the closing of four structural gaps in current shopping assistants — preference continuity, decision-critical attribute visibility, faithful explanations, and post-purchase lifecycle support. The business impact follows: in a domain where ~70% of carts are abandoned [5], a system that meaningfully reduces decision friction translates directly into recovered revenue and reduced post-purchase regret.

Three directions are natural extensions. (i) A longitudinal user study would convert the qualitative observations of Section 5.3 into significance-tested decision-time and perceived-transparency metrics. (ii) Replacing the static lifecycle dashboard with real calendar/kitchen integrations would convert Scene 5 from a demo into a product. (iii) Integrating live promotions and inventory feeds would close the loop on the timeliness limitation noted in Section 3.4.

7. Contributions

7.1 Team Members

This capstone is completed by a single student. Chenghui Tan is the sole author and is responsible for end-to-end design, implementation, and evaluation of the system: system architecture and the trust-boundary decision; data engineering (the Playwright scrapers, normaliser, and product schema); the ranking engine, explainability pass, and fallback logic; the FastAPI back-end and the React five-scene front-end; the scenario scripts and demo evaluation in Section 5; and this written report.

7.2 AI Tool Contributions

Three categories of AI assistance were used during the project.

- Claude (Anthropic, Opus 4.6) is part of the running system itself: it parses initial user input into structured preferences, maps free-text answers, and writes per-product explanations grounded in the rule-matches list. The trust-boundary discipline (Section 4.1) confines Claude to language work; ranking is never an LLM call.
- Claude Code (the agent CLI) was used as an authoring assistant during implementation: scaffolding components, drafting test cases, and pair-coding the recommendation engine. All AI-generated code was reviewed line-by-line, refactored, and committed under the author's name. The five-layer architecture, trust-boundary commitment, scenario design, and report structure are the author's intellectual contributions.
- ChatGPT (OpenAI) was used twice during literature review for structured search and for double-checking IEEE citation formatting. No model output was committed verbatim.

All AI contributions are disclosed in the spirit of the project's own transparency norm: the same explainability discipline that the system applies to its recommendations is applied here to its construction.

8. References

[1] P. Resnick and H. R. Varian, “Recommender systems,” Communications of the ACM, vol. 40, no. 3, pp. 56–58, 1997.

[2] G. Adomavicius and A. Tuzhilin, “Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions,” IEEE Transactions on Knowledge and Data Engineering, vol. 17, no. 6, pp. 734–749, 2005.

[3] M. de Gemmis et al., “Semantics-aware content-based recommender systems,” in Recommender Systems Handbook, Springer, 2015, pp. 119–159.

[4] J. Nielsen, “Search: Visible and simple,” Nielsen Norman Group, 2015. [Online]. Available: https://www.nngroup.com/articles/search-visible-simple/

[5] Baymard Institute, “49 Cart abandonment rate statistics,” Baymard Institute, 2024. [Online]. Available: https://baymard.com/lists/cart-abandonment-rate

[6] D. Kahneman, Thinking, Fast and Slow. New York, NY, USA: Farrar, Straus and Giroux, 2011, ch. on choice fatigue.

[7] R. C. Reardon, J. P. Sampson, and G. W. Peterson, “Career interventions and post-purchase regret,” Journal of Career Development, vol. 26, no. 3, pp. 197–211, 2000.

[8] Amazon, “Amazon Rufus product overview,” Amazon Inc., 2024. [Online]. Available: https://www.aboutamazon.com/news/retail/amazon-rufus

[9] OpenAI, “Introducing ChatGPT shopping,” OpenAI Blog, Nov. 2024. [Online]. Available: https://openai.com/blog/chatgpt-shopping

[10] X. Chen et al., “Towards conversational recommendation: A survey,” ACM Computing Surveys, vol. 55, no. 9, pp. 1–37, 2023.

[11] B. Shneiderman, Designing the User Interface: Strategies for Effective Human–Computer Interaction, 6th ed. Boston, MA, USA: Pearson, 2016.

[12] Perplexity AI, “Perplexity Pages and shopping,” Perplexity AI, 2024. [Online]. Available: https://www.perplexity.ai

[13] F. Ricci, L. Rokach, and B. Shapira, Eds., Recommender Systems Handbook, 3rd ed. New York, NY, USA: Springer, 2022.

[14] Y. Zhang and X. Chen, “Explainable recommendation: A survey and new perspectives,” Foundations and Trends in Information Retrieval, vol. 14, no. 1, pp. 1–101, 2020.

[15] T. Miller, “Explanation in artificial intelligence: Insights from the social sciences,” Artificial Intelligence, vol. 267, pp. 1–38, 2019.

[16] Anthropic, “Claude Opus 4.6 model card,” Anthropic, 2026. [Online]. Available: https://www.anthropic.com

[17] Microsoft Playwright, “Playwright for Python documentation,” Microsoft Corp., 2024. [Online]. Available: https://playwright.dev/python/

[18] R. T. Fielding and J. Reschke, “Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content,” IETF RFC 7231, 2014.

9. Appendix — Code Listings

9.1 ETL Pipeline Orchestrator (run_pipeline.py — excerpt)

```
def run_pipeline(categories: list[str]) -> dict:
    """Drive scrapers, then normalize. Always continues on per-source
failure."""
    raw = {}
    for cat in categories:
        for src in SOURCE_BY_CATEGORY[cat]:
            try:
                raw.setdefault(cat, []).extend(SCRAPERS[src](cat))
            except Exception as e:
                logger.warning("scraper %s/%s failed: %s", src, cat, e)
                continue
    return normalize.run(raw)
```

Listing 1: Top-level pipeline orchestration. Scrapes each category from each source, tolerates per-source failures, then defers to the normalizer for canonicalization.

9.2 Delivery Normalisation (normalize.py — excerpt)

```
def parse_delivery(text: str | None) -> int | None:
    """'Get it Tue, May 21' → days from today, else None."""
    if not text:
        return None
    m = DATE_RE.search(text)
    if not m:
        return None
    target = parse_date(m.group(1))
    return max(0, (target - date.today()).days)
```

Listing 2: Delivery-string parser. Returns None on unparseable text; the ranker treats None as a neutral 0.5 sub-score so missing data does not bias ranking.

9.3 Constraint-Aware Ranking (recommendation_engine_refactored.py — excerpt)

```
def recommend_products(prefs: dict, top_n: int = 10) -> list[dict]:
    """Filter → rank → top_n; relax constraints if too few survive."""
    products = load_products()

    result = _try_ranked(products, prefs, top_n)
    if result is not None:                      # full constraints
        return result

    # Fallback 1: drop delivery
    relaxed = {k: v for k, v in prefs.items() if k != "delivery_days_max"}
    result = _try_ranked(products, relaxed, top_n)
    if result is not None:
        return result

    # Fallback 2: drop price + delivery (category only)
    relaxed = {k: v for k, v in prefs.items()
               if k not in ("delivery_days_max", "price_max")}
```

```
    result = _try_ranked(products, relaxed, top_n)
    if result is not None:
        return result

    # Fallback 3: ignore category — top by shared score
    base = {"priority": prefs.get("priority", "balanced")}
    return rank_products(load_products(), base)[:top_n]
```

Listing 3: Three-tier fallback. Each tier relaxes one constraint; the ranker never returns an empty list as long as the dataset is non-empty.

9.4 Scoring Rule (water_bottle gym example)

```
def _score_water_bottle(product, prefs, features):
    score, matches = 0.0, []
    if prefs.get("use_case") == "gym":
        score = _apply_rule(score, matches,
            features["lightweight"] or _title_has(product, "freesip", "owala"),
            "gym_suitable")
        score = _apply_rule(score, matches, features["insulated"],
"insulated_gym")
    if prefs.get("insulated"):
        score = _apply_rule(score, matches, features["insulated"],
            "insulated", penalty=_SOFT_PENALTY)
    return score, matches
```

Listing 4: Category-specific scoring for the water-bottle gym use case. Bonuses are +0.10 per matched rule; soft penalties are −0.05 per missed rule the user requested.

9.5 FastAPI Routes (main.py — excerpt)

```
@app.post("/session/start")
def start_session(body: StartBody):
    try:
        parsed = claude_client.parse_initial_input(body.text)
        category = parsed.get("category", "unknown")
        preferences = parsed.get("preferences", {})
    except Exception:
        category, preferences = "unknown", {}     # graceful degradation

    if category in ("unknown", ""):
        return {"session_id": s["session_id"], "category": None,
                "next_question": None, "chips": CATEGORY_CHIPS}
    return {"session_id": s["session_id"], "category": category,
            "next_question": _next_question(s), "chips": None}
```

Listing 5: Session-start endpoint. The try/except keeps the demo functional when no Anthropic API key is available — chip selection becomes the fallback path.

9.6 Trust Boundary in the Recommendation Pipeline

```
def _run_recommendations(session: dict) -> list[dict]:
    prefs = {**session["preferences"], "category": session["category"]}
    products = engine.recommend_products(prefs, top_n=12)   # deterministic
    try:
```

```
            explanations = claude_client.write_explanations(
                products, session["preferences"], session["raw_input"])
        except Exception:
            explanations = [p.get("_explanation", "") for p in products]  #
    fallback
        return [_format_product(p, explanations[i]) for i, p in
    enumerate(products)]
```

Listing 6: Where the trust boundary lives in code. Ranking is a pure function call; explanation generation is best-effort and gracefully falls back to the deterministic explainer.